

嵌入式常见问题

一、C语言常见问题

1. 关键字static的作用是什么？

static关键字有两方面的用途，分别取决于它是在变量声明还是函数声明中使用。

static可以用于局部变量、全局变量以及函数声明。

1. 在局部变量中的作用

当static用于函数内的局部变量时，该变量成为静态局部变量：

它的生命周期与程序相同，它的值保留到程序结束

它被初始化一次，在程序开始执行时。如果未初始化，则默认为0或空字符串（对于数组）

每次函数被调用时，它会保留上次调用后的值

2. 在全局变量中的作用

当static用于全局变量时，它限制了变量的作用域。这样的变量只在定义它的源文件中可见，不能被其他源文件访问

3. 在函数声明中的作用

当static用于函数前，表示该函数是静态函数，其作用域仅限于定义它的源文件

注意：从作用域和生命周期2个维度解释清楚变量和函数的作用就行了

2. const关键字有什么作用/什么情况下使用？

1. 常量定义、限制变量被修改：

`const` 可以用于定义不会改变的变量，这样的变量被称为常量。一旦赋值，其值就不能再被修改。

```
const int MAX_SIZE = 100;
```

2. 指针修饰：

`const` 可以与指针结合使用，有三种基本形式：

- `const int *ptr;` 表示 `ptr` 指向的整数是常量，不能通过 `ptr` 修改。
- `int * const ptr = &var;` 表示 `ptr` 本身是一个常量指针，不能重新赋值给另一个地址。
- `const int * const ptr = &var;` 结合了上述两者，既不能修改 `ptr` 指向的内容，也不能改变 `ptr` 指向的地址。

3. 函数参数保护：

- 当函数接收一个 `const` 参数时，该函数不能修改传递给它的参数的值。这对于保证函数不破坏外部数据很有帮助。

核心答复：C语言中 `const` 关键字的作用是标记一个变量、指针或数组的值不可改变。

3.volatile关键字的作用？

(1) 裸机编程时，某变量是指向寄存器中某一特定地址，添加 `volatile` 的变量不进行优化处理；

(2) 某函数与中断函数共享全局变量时，加上 `volatile`，让编译器不要省略该变量的访问；

(3) 多线程中修饰共享全局变量，让编译器不要省略该变量的访问。

核心答复：volatile指定的关键字可能被系统、硬件、进程/线程改变，强制编译器每次从内存中取得该变量的值，而不是从被优化后的寄存器中读取。例子:硬件时钟;多线程中被多个任务共享的变量等。

4. sizeof和strlen的区别？

sizeof是一个运算符和关键字，但不是函数，用于计算数据类型或者变量所占用的字节数：

- sizeof可以应用于类型名或变量名。
- 对于数组，sizeof返回整个数组的大小，而非单个元素的大小。
- sizeof的值在编译期就已经确定，因此它可用于定义数组大小或常量。

strlen是一个标准库函数，用于计算以空字符'\0'结尾的字符串的长度，不包括这个终止符。

- strlen只能应用于字符数组或指向字符串的指针。
- strlen函数在运行时计算字符串的长度。
- 如果字符串没有正确终止，strlen可能会导致无限循环或访问越界。

核心答复：

sizeof：

- 是运算符和关键字，返回对象或类型所占用的字节数。包括空字符和其他数据。

strlen：库函数，返回字符串的有效长度（不包括空终止符）

5. ++a和a++有什么区别？

a++为：先使用a的值，然后再增加。

具体运算过程为：

```
int temp = a;  
  
a=a+1;  
  
return temp;
```

++a为：先增加a的值，然后再使用。

具体运算过程为：

```
a=a+1;  
return a;
```

6. 堆与栈有什么区别？

申请方式

栈的空间由操作系统自动分配/释放，堆上的空间手动分配/释放，

申请大小的限制

栈空间有限，如果申请的空间超过栈的剩余空间时，将提示overflow。因此，能从栈获得的空间较小，主要用于存储函数的局部变量、函数参数以及返回地址等

堆是很大的自由存储区。堆是向高地址扩展的数据结构，是不连续的内存区域。这是由于系统是用链表来存储的空闲内存地址的，自然是不连续的，而链表的遍历方向是由低地址向高地址。堆的大小受限于计算机系统中有效的虚拟内存。由此可见，堆获得的空间比较灵活，也比较大。

申请效率

栈由系统自动分配，速度较快。但程序员是无法控制的。

堆是由new/malloc分配的内存，一般速度比较慢，而且容易产生内存碎片,不过用起来最方便

7. 什么是内存泄露？

简单地说就是申请了一块内存空间，使用完毕后没有释放掉。

它的一般表现方式是程序运行时间越长，占用内存越多，最终用尽全部内存,整个系统崩溃。由程序申请的一块内存，且没有任何一个指针指向它，那么这块内存就泄露了。

8. 指针数组和数组指针的区别？

数组指针就是指向数组的指针，它表示的是一个指针，这个指针指向的是一个数组，它的重点是指针。

例如，`int(*pa)[8]`声明了一个指针，该指针指向了一个有8个int型元素的数组。

指针数组表示的是一个数组，而数组中的元素是指针，例如`int *p[4];`，包含了4个指针元素。

9. 函数指针和指针函数有什么区别？

1.函数指针

指向函数地址的指针。

如果在程序中定义了一个函数，那么在编译时系统就会为这个函数代码分配一段存储空间，这段存储空间的首地址称为这个函数的地址。而且函数名表示的就是这个地址。既然是地址我们就可以定义一个指针变量来存放，这个指针变量就叫作函数指针变量，简称函数指针。例如：`int(*p)(int,int);`

2. 指针函数

函数返回值类型是一个指针。

首先它是一个函数，只不过这个函数的返回值是一个地址值。函数返回值必须用同类型的指针变量来接受，也就是说，指针函数一定有“函数返回值”，而且，在主调函数中，函数返回值必须赋给同类型的指针变量。

例如：`int *pfun(int, int);`

10. 数组名和指针的区别是什么？

1. 数组：存储相同类型的元素。具有固定的大小。名字通常被视为指向首元素的常量指针。

2. 指针：存储内存地址。可以指向任何类型的数据。可以重新赋值以指向不同的地址。

主要区别在于：数组名不可改变指向的位置，而指针可以改变指向的位置。

11. 数组和链表的区别是什么？

数组：连续内存，大小固定，通过下标快速访问，但插入删除效率低。

链表：动态内存，大小可变，通过指针访问，插入删除效率高，但访问较慢。

12. 野指针是什么？

1.野指针是指向不可用内存的指针

当指针被创建时,指针不可能自动指向NULL，这时，默认值是随机的，此时的指针成为野指针。

2.当指针被free或delete释放掉时,如果没有把指针设置为NULL,

则会产生野指针，因为释放掉的仅仅是指针指向的内存，并没有把指针本身释放掉。

3.第三个造成野指针的原因是指针操作超越了变量的作用范围。

13. typedef 和#define的区别？

1.原理不同

#define是C语言中定义的**语法**,它是**预处理指令**,在预处理时进行**简单而机械的字符串替换**，不做正确性检查.

typedef是**关键字**，它在编译时处理,所以 typedef具有类型检查的功能。typedef用来定义类型的别名，这些类型不仅包含内部类型(int、char等),还包括自定义类型(如struct)，可以起到使类型易于记忆的功能。

例如: `typedef int(*PF)(const char *,const char *)`

定义一个指向函数的指针的数据类型PF,其中函数返回值为int，参数为 `const char *`;

2、作用域不同

typedef：作用于类型，创建的新类型可以在作用域内像其他类型一样使用。

#define：作用于文本，定义的宏在预处理阶段进行文本替换，不受作用域限制，直到遇到 `#undef` 才会失效。

14. #include <f.h>和#include "f.h"有什么区别？

对于# include <f.h>,编译器先从标准库路径开始搜索 f.h, 使得系统文件调用较快。

而对于#include "f.h",编译器先从用户的工作路径开始搜索 f.h, 然后去寻找系统路径, 使得自定义文件较快。

15.宏定义笔试题

写一个标准宏MIN/MAX, 这个宏输入两个参数并返回较小/较大的一个？

```
#define MIN(A , B)    ( (A) <= (B) ? (A) : (B) )  
  
#define MAX(A , B)    ( (A) >= (B) ? (A) : (B) )
```

16. 两数交换笔试题

交换两个变量的值, 不使用第三个变量。即a=3,b=5,交换之后a=5,b=3

有两种解法, 一种用算术算法, 一种用^(异或)。

```
a= a + b;  
b=a-b;  
a=a-b;
```

或者


```
a = a^b; // 只能对int,char..  
b = a^b;  
a = a^b;
```

17. 位运算笔试题

给定一个整型变量a，写两段代码，第一个设置a的bit3，第二个清除a的bit 3。在以上两个操作中，要保持其它位不变。

```
#define BIT3 (0x1 << 3)  
static int a;  
  
void set_bit3(void)  
{  
    a |= BIT3 ;  
}  
  
void clear_bit3(void)  
{  
    a &= BIT3 ;  
}
```

PS知识点：位运算/位操作

18.C语言中结构体 struct与 联合体union的区别是什么？

- struct/结构体：
 - 成员变量在内存中按顺序排列。
 - 各成员占用独立的内存空间。
 - 总体大小等于各成员大小之和。
- union/联合体：

- 所有成员共享同一段内存。
- 只能同时存储一个成员的值。
- 总体大小等于最大成员的大小。

19.指针常量，常量指针，指向常量的常量指针有什么区别？

1.指针常量

```
int * const p;
```

先看const再看*，p是一个常量类型的指针，**不能修改这个指针的指向**，但是这个指针所指向的地址上存储的值可以修改。

2.常量指针

```
const int *p;  
int const *p;
```

先看*再看const，定义一个指针指向一个常量，**不能通过指针来修改这个指针指向的值**

3.指向常量的常量指针

```
const int *const p
```

对于“指向常量的常量指针”，就必须同时满足上述1和2中的内容，既不可以修改指针的值，也不可以修改指针指向的值。

20. 关于全局变量

全局变量和静态变量的区别是什么？

1. **全局变量的作用域为程序块，而局部变量的作用域为当前函数。**
2. **内存存储方式不同**,全局变量(静态全局变量,静态局部变量)分配在全局数据区(静态存储空间)，后者分配在栈区。
3. **生命周期不同**。全局变量随主程序创建而创建，随主程序销毁而销毁,局部变量在局部函数内部甚至局部循环体等内部存在，退出就不存在了。
4. **使用方式不同**。通过声明为全局变量，程序的各个部分都可以用到，而局部变量只能在局部使用。

局部变量能否和全局变量重名？

能，局部会屏蔽全局。局部变量可以与全局变量同名，在函数内引用这个变量时，会用到同名的局部变量，而不会用到全局变量。

使用全局变量需要注意什么？

1. **命名冲突：**
 - 全局变量名称可能与其他模块中的局部变量冲突。
2. **可见性问题：**
 - 全局变量在整个程序中可见，可能导致意外修改。
3. **初始化问题：**
 - 未初始化的全局变量可能导致不确定的行为。
4. **内存占用：**
 - 全局变量会增加程序的内存占用。
5. **多线程安全问题：**

- 在多线程环境中，全局变量可能引发竞态条件。

6. 代码可维护性降低：

- 过度使用全局变量会使代码难以理解和维护。

7. 资源泄漏：

- 如果全局变量持有需要释放的资源，可能会导致资源泄漏。

8. 调试难度增加：

- 全局变量的变化难以追踪，增加了调试难度。

为了减少这些问题，建议尽量限制全局变量的使用，优先考虑使用局部变量或通过函数参数传递所需的数据。

21. extern关键字的用途是什么？

- `extern` 关键字用于声明一个已经在其他文件中定义过全局变量或函数。它告诉编译器变量的实际定义在其他地方，链接时将其其他文件的内容链接到当前文件中。

22. .h头文件中的 ifndef/define/endif 的作用？

防止该头文件被重复引用。

23. 队列和栈有什么区别？

队列先进先出，栈后进先出

24. do{} while 和 while{}有什么区别?

前一个循环一遍再判断，后一个判断以后再循环。

25.如何引用一个已经定义过全局变量?

使用extern关键字

26.一个32位的机器,该机器的指针是多少位

指针是多少位只要看地址总线的位数就行了。所以指针的位数就是4个字节了 (32/8)

对应：64位就是64/8=8个字节

27.inline内联函数

1. 定义：

内联函数是一种特殊的函数，它的调用会被编译器替换为函数体本身的代码，类似于宏替换，但更加安全。

inline关键字仅仅是建议编译器做内联展开处理，即是将函数直接嵌入调用程序的主体，省去了调用/返回指令。

2. 语法：

- 在函数定义前加上 `inline` 关键字来声明一个内联函数。
- 例如：

C深色版本

```
1 inline int add(int x, int y) {  
2     return x + y;  
3 }
```

3. 优势：

- 减少了函数调用的开销（如保存和恢复寄存器状态）。
- 提高了程序的执行速度。

4. 限制：

- 内联函数的使用受到编译器的限制，编译器可以选择是否真正内联函数。
- 过度使用内联函数可能导致代码膨胀，增加程序的内存占用。

28.C语言结构体对齐问题

题目：假设有一个结构体定义如下：

```
struct MyStruct {  
    char c;  
    int i;  
    double d;  
};
```

结构体 `MyStruct` 中包含了一个 `char` 类型，一个 `int` 类型和一个 `double` 类型的成员。请问在32位系统中该结构体的大小是多少？

解答：在32位系统中，结构体的对齐通常遵循以下规则：

1. 每个成员分配在自然对齐的地址上。
2. 结构体整体有一个对齐的要求，通常是最宽基本类型成员的大小。

3. 如果结构体中包含嵌套结构体，嵌套结构体也要对齐到自己最大成员的大小。

对于 `MyStruct`，`char` 类型成员 `c` 没有特殊的对齐要求，`int` 类型成员 `i` 在32位系统中是4字节对齐，`double` 类型成员 `d` 在32位系统中是8字节对齐。

结构体的大小需要满足两个条件：

1. 每个成员按自己的对齐要求放置。
2. 结构体整体对齐要求满足最大成员的对齐要求。

根据上述规则，`MyStruct` 的大小至少应为 `char` (1字节) + `int` (4字节) + `double` (8字节) = 13字节。但是，实际上由于对齐的原因，结构体的大小至少应为16字节（最大成员 `double` 的大小）。

因此，在32位系统中，结构体 `MyStruct` 的大小应该是16字节。

29.冒泡排序

基本思想：

把第一个元素与第二个元素比较，如果第一个比第二个大，则交换他们的位置。接着继续比较第二个与第三个元素，如果第二个比第三个大，则交换他们的位置。我们对每一对相邻元素作同样的工作，从开始第一对到结尾的最后一对，这样一趟比较交换下来之后排在最右的元素就会是最大的数。除去最右的元素，我们对剩余的元素做同样的工作，如此重复下去直到排序完成。

具体步骤：

- 1.比较相邻的元素。如果第一个比第二个大，就交换他们两个。
- 2.对每一对相邻元素作同样的工作,从开始第一对到结尾的最后一对。这步做完后，最后的元素会是最大的数。

3.针对所有的元素重复以上的步骤，除了最后一个

4.持续每次对越来越少的元素重复上面的步骤，直到没有任何一对数字需要比较。

代码：

```
void bubblesort(int *arr,int n)
{
    int m,i,j;
    for(i=0;i<n-1;i++)
    {
        for(j=0;j<n-1-i;j++)
        {
            if(arr[j]>arr[j+1])
            {
                m=arr[j];
                arr[j]=arr[j+1];
                arr[j+1]=m;
            }
        }
    }
}
```

30.选择排序

基本思想

首先，找到数组中最小(/最大)的那个元素，其次，将它和数组的第一个元素交换位置(如果第一个元素就是最小元素那么它就和自已交换)。其次，在剩下的元素中找到最小的元素，将它与数组的第二个元素交换位置。如此往复，直到将整个数组排序。这种方法我们称之为选择排序。

具体步骤

1.首先在未排序序列中找到最小(大)元素，存放到排序序列的起始位置。

2. 再从剩余未排序元素中继续寻找最小(大)元素，然后放到已排序序列的末尾。
3. 重复第二步，直到所有元素均排序完毕。

代码

```
#include <stdio.h>

void selectionSort(int arr[], int n) {
    int i, j, minIndex, temp;

    // One by one move boundary of unsorted subarray
    for (i = 0; i < n - 1; i++) {
        // Find the minimum element in unsorted
        array
        minIndex = i;
        for (j = i + 1; j < n; j++)
            if (arr[j] < arr[minIndex])
                minIndex = j;

        // Swap the found minimum element with the
        first element
        temp = arr[minIndex];
        arr[minIndex] = arr[i];
        arr[i] = temp;
    }
}

int main() {
    int arr[] = {64, 25, 12, 22, 11};
    int n = sizeof(arr) / sizeof(arr[0]);

    selectionSort(arr, n); //选择排序核心算法

    printf("Sorted array: \n");
    for (int i = 0; i < n; i++)
```

```
        printf("%d ", arr[i]);  
        printf("\n");  
  
        return 0;  
    }
```

31.C程序的内存结构

从上到下的角度来看，C程序的内存结构可以分为以下几个主要部分：

1. 代码段 (Text Segment 或 Code Segment)：

- 存储程序的机器指令。
- 通常是只读的。

2. 数据段 (Data Segment)：

- 包含程序中的全局变量和静态变量。
- 又分为初始化数据段和未初始化数据段。
- 初始化数据段存放初始化过的全局变量和静态变量。
- 未初始化数据段存放未初始化的全局变量和静态变量，默认值通常为0。

3. BSS段 (Block Started by Symbol)：

- 存放未初始化的全局变量和静态变量。
- 这些变量默认初始化为0。

4. 堆 (Heap)：

- 动态分配的内存区域。
- 由程序员通过函数如 `malloc`, `calloc`, `realloc`, 和 `free` 控制分配和释放。

5. 栈 (Stack)：

- 用于存储函数调用时的局部变量和函数参数。
- 栈遵循后进先出（LIFO）原则。
- 栈内存的分配和释放是连续的，由编译器自动管理。
-

32.什么是大端?什么是小端?

大端模式:低位字节存在高地址上，高位字节存在低地址上。

小端模式:高位字节存在高地址上，低位字节存在低地址上。

二、Linux系统编程常见问题

1.什么是进程?什么是线程?

进程是资源分配的基本单位，它是程序执行时的一个实例，在程序运行时创建。

线程是程序执行的最小单位，是进程的一个执行流，一个线程由多个线程组成的。

2.进程和线程有什么区别?

1.进程是资源分配的最小单位。

2.线程是程序执行的最小单位,也是处理器调度的基本单位，但进程不是，两者均可并发执行。

3.进程有自己的独立地址空间,每启动一个进程，系统就会为它分配地址空间，建立数据表来维护代码段、堆栈段和数据段，这种操作非常昂贵。而线程是共享进程中的数据，使用相同的地址空间，因此，CPU切换一个线程的花费远比进程小很多，同时创建一个线程的开销也比进程小很多。

4.线程之间的通信更方便,同一进程下的线程共享全局变量、静态变量等数据,而进程之间的通信需要以通信的方式(IPC)进行。不过如何处理好同步与互斥是编写多线程程序的难点。但是多进程程序更健壮,多线程程序只要有一个线程死掉,整个进程也跟着死掉了,而一个进程死掉并不会对另外一个进程造成影响,因为进程有自己独立的地址空间。

5.进程切换时,消耗的资源大,效率低。所以涉及到频繁的切换时,使用线程要好于进程。同样如果要求同时进行并且又要共享某些变量的并发操作,只能用线程不能用进程。

6.执行过程:每个独立的进程有一个程序运行的入口、顺序执行序列和程序入口。但是**线程不能独立执行**,必须依存在应用程序中,由应用程序提供多个线程执行控制。

7.线程执行开销小,但是不利于资源的管理和保护。线程适合在SMP机器(双CPU系统)上运行。进程执行开销大,但是能够很好的进行资源管理和保护,可以跨机器迁移。

核心回复:

- 进程:操作系统分配资源的最小单元,独立资源分配,独立地址空间,开销大,稳定但通信复杂。
- 线程:线程是程序执行的最小单位,也是处理器调度的基本单位,共享进程资源,开销小,通信直接,但同步复杂,易影响整体进程。

3.什么时候使用多进程, 什么时候使用多线程?

对资源的管理和保护要求高,不限制开销和效率时,使用多进程。

要求效率高,频繁切换时,资源的保护管理要求不是很高时,使用多线程。

4.进程有几种状态?

进程可以分为五个状态，分别是：

- 1.创建状态
- 2.就绪状态
- 3.运行状态
- 4.阻塞状态
- 5.终止状态

5.Linux进程间通信有几种?

管道、信号、消息队列、共享内存、信号量和套接字

上述几种方式的优缺点

1.(无名)管道 (Pipe)

- 特点：
 - 简单的通信机制，实现起来容易。
 - 可以实现进程间的单向数据流。
 - 适用于父子进程之间的通信。
- 优点：
 - 简单易用，实现起来容易。
 - 适用于小量数据的传输。
 - 不需要额外的同步机制。

- 缺点：
 - 仅限于单向通信。
 - 两端必须是父子进程或具有父子关系的进程。
 - 管道缓冲区有限，一般不超过4KB。
- 适用场景：
 - 简单的数据传输。
 - 父子进程之间的通信。

2.命名管道 (Named Pipe)

- 特点：
 - 可以在无亲缘关系的进程之间通信。
 - 支持文件系统，因此可以持久化通信。
 - 可以实现双向通信。
- 优点：
 - 可以在无亲缘关系的进程之间通信。
 - 支持文件系统，因此可以持久化通信。
 - 可以实现双向通信。
- 缺点：
 - 相比普通管道，开销稍大。
 - 仍然受限于单向通信的特性，尽管可以通过创建两个命名管道来实现双向通信。
- 适用场景：
 - 无亲缘关系的进程之间的通信。
 - 需要持久化通信的情况。

3. 消息队列 (Message Queues)

- 特点：
 - 支持复杂的数据结构。
 - 可以在多个进程中传递不同类型的消息。
 - 消息队列提供了高级别的抽象，使得进程间通信更加灵活。
- 优点：
 - 支持复杂的数据结构。
 - 可以在多个进程中传递不同类型的消息。
 - 消息队列提供了高级别的抽象，使得进程间通信更加灵活。
- 缺点：
 - 实现较复杂。
 - 开销较大，不适合大量数据的传输。
 - 需要额外的同步机制来确保消息的正确处理。
- 适用场景：
 - 需要在多个进程中传递不同类型的消息。
 - 需要支持复杂的数据结构。

4. 信号 (Signals)

- 特点：
 - 用于通知进程发生特定事件。
 - 简单，适用于简单的进程间通知。
 - 可以立即响应特定事件。
- 优点：
 - 用于通知进程发生特定事件。
 - 简单，适用于简单的进程间通知。
 - 可以立即响应特定事件。

- 缺点：
 - 信号处理可能复杂。
 - 不能传递大量数据。
 - 信号的默认处理行为可能不是预期的，需要明确处理信号。
- 适用场景：
 - 用于简单的进程间通知。
 - 需要立即响应特定事件的情况。

5. 信号量 (Semaphores)

- 特点：
 - 用于进程间的同步。
 - 可以避免竞态条件。
 - 适用于保护共享资源。
- 优点：
 - 用于进程间的同步。
 - 可以避免竞态条件。
 - 适用于保护共享资源。
- 缺点：
 - 不适合数据传输。
 - 实现较为复杂，需要理解同步原理。
 - 错误使用可能导致死锁。
- 适用场景：
 - 用于进程间的同步。
 - 需要避免竞态条件的情况。

6. 共享内存 (Shared Memory)

- 特点：
 - 高效的数据传输。
 - 直接访问内存中的数据，减少上下文切换和数据复制的开销。
 - 可以在多个进程中共享大量数据。
- 优点：
 - 高效的数据传输。
 - 直接访问内存中的数据，减少上下文切换和数据复制的开销。
 - 可以在多个进程中共享大量数据。
- 缺点：
 - 需要额外的同步机制来避免数据竞争。
 - 实现较为复杂，需要正确管理内存的生命周期。
 - 错误使用可能导致数据不一致。
- 适用场景：
 - 需要在多个进程中共享大量数据。
 - 需要高效的数据传输。

7. 套接字 (Sockets)

- 特点：
 - 适用于网络通信和本地进程间通信。
 - 支持多种通信模式（流式、数据报）。
 - 可以在任意两个进程之间建立通信。
- 优点：
 - 适用于网络通信和本地进程间通信。
 - 支持多种通信模式（流式、数据报）。

- 可以在任意两个进程之间建立通信。
- 缺点：
 - 相比其他IPC机制，开销较大。
 - 实现相对复杂，需要理解网络编程的概念。
 - 需要额外的同步机制来确保数据的正确传输。
- 适用场景：
 - 适用于网络通信和本地进程间通信。
 - 需要支持多种通信模式。

8. POSIX消息队列 (POSIX Message Queues)

- 特点：
 - 支持复杂的数据结构。
 - 可以在多个进程中传递不同类型的消息。
 - 更现代的标准，提供了更好的安全性和可靠性。
- 优点：
 - 支持复杂的数据结构。
 - 可以在多个进程中传递不同类型的消息。
 - 更现代的标准，提供了更好的安全性和可靠性。
- 缺点：
 - 实现较复杂。
 - 开销较大，不适合大量数据的传输。
 - 需要额外的同步机制来确保消息的正确处理。
- 适用场景：
 - 需要在多个进程中传递不同类型的消息。
 - 需要支持复杂的数据结构。

6.Linux进程间通信效率最高的是哪一种？为什么？

共享内存。因为它允许多个进程直接访问同一块内存，而无需进行数据拷贝。同时**没有内核态和用户态之间的切换开销。**

7.使用共享内存有什么注意点？

- 1.**同步与互斥：**避免多个进程同时写入，使用信号量或互斥锁等同步机制来保持数据一致性。
- 2.**内存管理：**合理分配和释放共享内存，避免内存泄漏和频繁分配销毁。如果一个进程崩溃或者没有及时解除共享内存映射，就有可能导致内存泄漏的问题。
- 3 **数据安全性：**共享内存是多个进程共享的，因此必须要注意数据的安全性和隐私性，避免敏感数据泄露。

8.共享内存和消息队列有什么区别？

1. **通信机制：**共享内存直接访问同一块内存；消息队列通过发送和接收消息进行通信。
2. **数据访问：**共享内存速度快但需同步处理；消息队列有额外开销但避免直接内存访问冲突。
3. **可靠性：**共享内存数据可能因崩溃丢失；消息队列通常提供持久化机制。
4. **适用场景：**共享内存适合高速大量数据交换；消息队列适合分布式、异步处理。
5. **实现复杂度：**共享内存同步处理复杂；消息队列提供高级别抽象，实现较简单但涉及系统调用。

9.线程间同步方法有哪些？

互斥锁、条件变量、信号量、自旋锁、读写锁、屏障（Barrier）

10.线程间互斥的方法有哪些？

互斥锁、条件变量、信号量、读写锁、

11.什么情况会导致死锁？

多个并发进程因争夺系统资源而产生相互等待的现象。即:一组进程中的每个进程都在等待某个事件发生，而只有这组进程中的其他进程才能触发该事件，这就称这组进程发生了死锁。

如：线程A按先加锁1再加锁2的顺序加锁，线程B按先加锁2再加锁1的顺序加锁，线程A和B都无法获得第二把锁，导致线程A与B甚至其他想获得锁的线程都会被阻塞而无法向前运行

12.如何避免死锁？

- **确保资源顺序的一致性**：所有线程在访问多个资源时，按照相同的顺序请求这些资源。
- **使用超时（Timeouts）**：在尝试锁定资源时设置超时，避免无限期等待。
- **避免嵌套锁**：尽量减少在持有锁时请求其他锁的情况。
- **使用锁尝试（Try Locks）**：使用 `pthread_mutex_trylock()` 代替 `pthread_mutex_lock()` 来尝试获取锁，避免阻塞。
- **资源一次性分配**：在程序开始时就分配好所有需要的资源，避免在运行时动态分配资源导致死锁。

- **死锁检测和恢复**：实现死锁检测机制，并在检测到死锁时采取相应的恢复措施。

13.什么是僵尸进程，孤儿进程，守护进程？

僵尸进程是一个进程使用fork创建子进程，如果子进程退出,而父进程并没有调用wait或waitpid获取子进程的状态信息，那么子进程的进程描述符仍然保存在系统中。这种进程称之为僵死进程。

孤儿进程是因为父进程异常结束了，然后被1号进程init收养。

守护进程是创建守护进程时有意把父进程结束，然后被1号进程init收养

孤儿进程和守护进程的区分：一个正常运行的子进程，如果此刻子进程退出，父进程没有及时调用wait或waitpid收回子进程的系统资源，该进程就是僵尸进程，如果系统收回了，就是正常退出，如果一个正常运行的子进程，父进程退出了但是子进程还在，该进程此刻是孤儿进程，被init收养，如果父进程是故意被杀掉，子进程做相应处理后就是守护进程。

14.僵尸进程有什么危害？

在进程退出的时候，内核释放该进程所有的资源,包括打开的文件，占用的内存等。但是仍然为其保留一定的信息(包括进程号 PID,退出状态 the termination status of the process,运行时间 the amount of CPU time taken by the process 等)。直到父进程通过wait/waitpid 来取时才释放。

如果进程不调用 wait/waitpid 的话，那么保留的那段信息就不会释放，其**进程号就会一直被占用，但是系统所能使用的进程号是有限的，如果大量的产生僵死进程，将因为没有可用的进程号而导致系统不能产生新的进程。**

15.如何清理僵尸进程？

僵尸进程的产生是因为父进程没有 `wait()` 子进程。所以如果我们自己写程序的话一定要在父进程中通过`wait()`来避免僵尸进程的产生。当系统中出现了僵尸进程时，我们是无法通过`kill` 命令把它清除掉的。但是我们可以杀死它的父进程让它变成孤儿进程，并进一步被系统中管理孤儿进程的进程收养并清理

三、网络编程

1.TCP如何保证可靠传输

连接管理：通过三次握手与四次挥手，保证连接实体真实存在

确认应答与序列号：TCP传输时将每个字节的数据都进行了编号，这就是序列号。

确认应答：TCP传输的过程中，每次接收方收到数据后，都会对传输方进行确认应答。也就是发送ACK报文。这个ACK报文当中带有对应的确认序列号，告诉发送方，接收到了哪些数据，下一次的数据从哪里发。

数据校验：TCP报文头有校验和，用于校验报文是否损坏

超时重传：如果发送一直收不到应答，可能是发送数据丢失，也可能是应答丢失，发送方再等待一段时间之后都会进行重传。

流量控制：当接收方来不及处理发送方的数据，能通过滑动窗口，提示发送方降低发送的速率，防止包丢失。

拥塞控制：网络层拥堵造成的拥塞，包括慢启动，拥塞避免，快速重传三种机制

2.TCP的连接和断开过程

三次握手

TCP (Transmission Control Protocol) 的三次握手 (Three-Way Handshake) 是建立TCP连接的过程。这一过程确保了双方都准备好通信，并且能够相互发送和接收数据。下面是TCP三次握手的详细步骤：

第一步：SYN (同步)

- **发起方** (Client) 向**接收方** (Server) 发送一个SYN (同步) 数据包，其中包含一个初始序列号 (ISN) 。
- **SYN标志位**被设置为1，序列号字段包含客户端选择的一个随机数 `seq=x`。

第二步：SYN-ACK (同步确认)

- **接收方** (Server) 接收到SYN数据包后，向**发起方** (Client) 发送一个SYN-ACK数据包。
- **SYN标志位**和**ACK标志位**都被设置为1。
- **序列号字段**包含服务器选择的一个随机数 `seq=y`。
- **确认号字段**包含客户端序列号加1的值 `ack=x+1`。

第三步：ACK (确认)

- **发起方** (Client) 接收到SYN-ACK数据包后，向**接收方** (Server) 发送一个ACK数据包。
- **ACK标志位**被设置为1。
- **序列号字段**包含客户端上一次使用的序列号加1的值 `seq=y+1`。
-

四次挥手TCP (Transmission Control Protocol) 的四次挥手

(Four-Way Wave) 是指TCP连接终止的过程。当一方希望关闭连接时，它会发送一个FIN (Finish) 标志位的数据包，另一方在收到后也会发送自己的FIN数据包以完成连接的关闭。下面是TCP四次挥手的详细步骤：

第一步：发送FIN（主动关闭）

- **主动关闭的一方**（通常称为Client）发送一个FIN数据包给**被动关闭的一方**（通常称为Server）。
- **FIN标志位**被设置为1。
- **序列号字段**包含最后一个数据包之后的序列号 `seq=u`。

第二步：发送ACK（确认FIN）

- **被动关闭的一方**（Server）接收到FIN数据包后，发送一个ACK数据包作为确认。
- **ACK标志位**被设置为1。
- **确认号字段**包含主动关闭的一方的序列号加1 `ack=u+1`。
- **序列号字段**包含被动关闭的一方当前的序列号 `seq=v`。

第三步：发送FIN（被动关闭）

- **被动关闭的一方**（Server）完成其数据发送后，发送自己的FIN数据包给**主动关闭的一方**（Client）。
- **FIN标志位**被设置为1。
- **序列号字段**包含被动关闭的一方发送的最后一个数据包之后的序列号 `seq=w`。

第四步：发送ACK（确认FIN）

- **主动关闭的一方**（Client）接收到FIN数据包后，发送一个ACK数据包作为确认。
- **ACK标志位**被设置为1。
- **确认号字段**包含被动关闭的一方的序列号加1 `ack=w+1`。
- **序列号字段**包含主动关闭的一方当前的序列号 `seq=u+1`。

3.TCP和UDP的区别是什么？

- 1.TCP 是面向连接的，UDP 是面向无连接的
- 2.UDP程序结构较简单
- 3.TCP 是面向字节流的，UDP 是基于数据报的协议
- 4.TCP 保证数据正确性，UDP 可能丢包，
- 5.TCP 保证数据顺序，UDP 不保证

4.TCP和UDP的优缺点是什么？

TCP的优点

可靠，稳定 TCP的可靠体现在TCP在传递数据之前，会有三次握手来建立连接，而且在数据传递时，有确认、窗口、重传、拥塞控制机制，在数据传完后，还会断开连接用来节约系统资源。

TCP的缺点

慢，效率低，占用系统资源高，易被攻击TCP在传递数据之前，要先建连接，这会消耗时间，而且在数据传递时，确认机制、重传机制、拥塞控制机制等都会消耗大量的时间，而且要在每台设备上维护所有的传输连接，事实上，每个连接都会占用系统的CPU、内存等硬件资源。而且，因为TCP有确认机制、三次握手机制，这些也导致TCP容易被人利用，实现DOS、DDOS、CC等攻击。

UDP的优点

快，UDP没有TCP的握手、确认、窗口、重传、拥塞控制等机制，UDP是一个无状态的传输协议，所以它在传递数据时非常快。没有TCP的这些机制，UDP较TCP被攻击者利用的漏洞就要少些。但UDP也是无法避免攻击的，比如:UDPFlood攻击。

UDP的缺点

不可靠，不稳定 因为UDP没有TCP那些可靠的机制，在数据传递时，如果网络质量不好，就会很容易丢包。

5.TCP和UDP的试用场景

TCP应用场景

效率要求相对低，但对准确性要求相对高的场景。因为传输中需要对数据确认、重发、排序等操作，相比之下效率没有UDP高。举几个例子:文件传输(准确高要求高、但是速度可以相对慢)、接受邮件、远程登录。

UDP应用场景

效率要求相对高，对准确性要求相对低的场景。举几个例子: qq聊天、在线视频、网络语音电话(即时通讯，速度要求高，但是出现偶尔断续不是太大问题,并且此处完全不可以使用重发机制)、广播通信(广播、多播)。

6.TCP相比UDP为什么 是可靠的?

存在：确认和重传机制，数据排序，流量控制、拥塞控制等机制

以上就是TCP比UDP传输更可靠的原因。

7.网络模型：网络OSI模型和TCP/IP模型

网络模型一般是指OSI七层参考模型和TCP/IP四层参考模型

OSI分层(7层): 物理层、数据链路层、网络层、传输层、会话层、表示层、应用层。

TCP/IP分层(4层): 网络接口层、网际层、运输层、应用层。

网络层: IP协议、ICMP协议、ARP协议、RARP协议。

传输层: UDP协议、TCP协议。

应用层: FTP(文件传送协议)、Telnet(远程登录协议)、DNS(域名解析协议)、SMTP(邮件传送协议), POP3协议(邮局协议), HTTP协议。

8.HTTP和HTTPS网络默认端口

HTTP默认端口:80

HTTPS默认端口:443

9.请你来说一下socket编程中服务器端和客户端主要用到哪些函数?

基于TCP的socket

1.服务器端程序

- (1)、创建一个socket, 用函数socket()
- (2)、绑定IP地址、端口等信息到socket上, 用函数bind()
- (3)、设置允许的最大连接数, 用函数listen()
- (4)、接收客户端上来的连接, 用函数accept()
- (5)、收发数据, 用函数send()和recv(), 或者read()和write()
- (6)、关闭网络连接

2.客户端程序:

- (1)、创建一个socket,用函数socket()
- (2)、设置要连接的对方的IP地址和端口等属性
- (3)、连接服务器, 用函数connect()
- (4)、收发数据, 用函数send()和recv(), 或read()和write()
- (5)、关闭网络连接

基于UDP的socket

1.服务器端流程

- (1)、建立套接字文件描述符，使用函数`socket()`，生成套接字文件描述符。
- (2)、设置服务器地址和侦听端口，初始化要绑定的网络地址结构。
- (3)、绑定侦听端口，使用`bind()` 函数，将套接字文件描述符和一个地址类型变量进行绑定。
- (4)、接收客户端的数据，使用`recvfrom()`函数接收客户端的网络数据。
- (5)、向客户端发送数据，使用`sendto()`函数向服务器主机发送数据。
- (6)、关闭套接字，使用`close()`函数释放资源。

2.客户端流程

- (1)、建立套接字文件描述符， `socket()`。
- (2)、设置服务器地址和端口， `struct sockaddr`。
- (3)、向服务器发送数据， `sendto()`。
- (3)、接收服务器的数据， `recvfrom()`。
- (5)、关闭套接字， `close()`。

10.请你来说一下GET和POST的区别

- 1.get参数通过url传递， post放在request body中。
- 2.get请求在url中传递的参数是有长度限制的，而post没有。
- 3.get比post更不安全，因为参数直接暴露在url中，所以不能用来传递敏感信息。
- 4.get请求只能进行url编码，而post支持多种编码方式。

5.get请求会浏览器主动cache,而post支持多种编码方式。

6.get请求参数会被完整保留在浏览历史记录里，而post中的参数不会被保留。

7.GET和POST本质上就是TCP链接，并无差别。但是由于HTTP的规定和浏览器/服务器的限制，导致他们在应用过程中体现出一些不同。

8.GET产生一个TCP数据包:POST产生两个TCP数据包

四、单片机（裸机部分）

1.STM32F103C8T6的基础参数信息

内核不同： F1是Cortex-M3内核

主频不同： F1主频72MHz

内存大小： F1内部SRAM最大64K

SRAM： 20 KB

ADC： 12 位 ADC，最多 16 通道

DAC： 2 个 12 位 DAC 通道

USART/UART： 2 个 USART 和 1 个 UART 接口

SPI： 2 个 SPI 接口

I2C： 2 个 I2C 接口

GPIO： 多达 48 个可编程 GPIO 引脚

CAN： 1 个 CAN 接口（STM32F103C8T6 不支持 CAN）

USB： 1 个 USB 2.0 全速接口（12 Mbps）

2.GPIO工作模式有几种，分别是哪些？

八种

浮空输入；上拉输入；下拉输入；模拟输入；

开漏输出；推挽输出；复用推挽；复用开漏。

3.推挽模式和开漏模式区别？

推挽输出：

- 特点：

可以**直接输出高低电平**，能够直接连接到电源和地，**无需外部上拉或下拉电阻**。

输出电压稳定，对信号的放大和传输有良好的保证。

具有**较高的输出电流能力**，适用于驱动各种负载，如电机、继电器等。

功耗较低，当输出处于静态状态时，只需要很小的工作电流。

适用场景：常用于数字电路中，实现电子设备的**开关功能**，如驱动LED、继电器等。

开漏输出：

- 特点：

默认只能输出低电平和高阻态，需要外部上拉电阻来提供高电平信号。

输出电压的高低由外部上拉电阻决定，对信号的放大和传输可能会有一定影响。

可以承受较高的输出电流，但具体电流大小受外部上拉电阻限制。

具有较强的抗干扰能力，适用于需要高抗干扰能力的负载。

适用场景：常用于接口电路中，如I2C、SPI、CAN等总线系统，以及LED控制、输入/输出（I/O）扩展等场景。多个开漏输出可以通过一个外部上拉电阻连接到同一总线上，实现多路设备的共享控制。

核心回复总结：

- **推挽输出：**可以直接输出高电平和低电平，适用于直接驱动负载，有较强的电流输出能力
- **开漏输出：**只能输出低电平或高阻态，需要外部上拉电阻，适用于需要多个设备共享信号线的场合。

4.通用定时器的功能有哪些？

基本定时功能、输入捕获、输出比较

5.USART的特点以及数据帧格式？

USART特点：异步 串行 全双工

UART数据帧格式为：起始位（0有效）-数据位（8位）-校验位（奇偶校验）-停止位（1位）

6.什么是波特率？

在串行通信中，波特率通常指的是每秒传输的位数。

7.115200波特率对应是多少字节？

115200波特率意味着每秒钟可以传输115200个位。在大多数串行通信协议中，一个字节（byte）由8位组成，即 $115200/8$ 个字节，

但在实际的串行通信中，还需要考虑起始位、停止位以及可能的奇偶校验位。

假设使用的是**最常用的8位数据位、1位起始位和1位停止位的配置**

（即8N1配置，其中N表示无奇偶校验位），那么一个完整的字节传输需要10位（ $1 + 8 + 1$ ）。因此，我们可以计算每秒传输的字节数如下：

每秒传输的字节数=波特率位/字节

对于**115200波特率**：

每秒传输的字节数= $115200/10=11520$ 字节/秒

8.RS232和RS485的区别？

RS232与RS485的主要区别如下：

1. **电气特性**：RS232是单端通信，信号电平高，易受干扰；RS485是差分通信，信号电平低，抗干扰能力强。
2. **通信模式**：RS-232：支持全双工通信。RS-485：默认支持半双工通信，可通过额外硬件实现全双工通信。
3. **传输距离**：RS232通常限于几十米内；RS485可达数千米。
4. **网络拓扑**：RS232主要是点对点；RS485支持多点通信，可采用总线结构。
5. **传输速率**：RS232常见115.2kbps；RS485最高可达10Mbps（实际受距离影响）。

6. **应用场景**：RS232常用于个人电脑与外设连接；RS485广泛用于工业自动化、楼宇自动化等需要长距离、多设备通信的场合。

9.I2C是如何进行通信的？

I²C 总线使用两条线：一条是串行数据线（SDA）和一条是串行时钟线（SCL）。下面是如何进行通信的简要说明：

1. **启动信号**：开始通信时，**SDA 线从高电平被拉低，此时 SCL 线必须保持高电平状态。**
2. **寻址**：发送器发送起始位后，接着发送目标设备的7位地址，最后发送一位表示读或写的指令位（R/W）。
3. **进行数据传输**：发送器发送数据位时，每个数据位后面跟着一个时钟脉冲。
4. **应答位**：接收器在接收到每个字节后发送一个应答位（ACK）来确认数据接收。**应答位是一个低电平脉冲。**
5. **停止信号**：通信结束时，SDA 线从**低电平被拉高**，此时 **SCL 线仍然保持高电平状态。**

示例

下面是一个简化的 I²C 通信示例：

1. **启动信号**：SDA 从高电平拉低，SCL 保持高电平。
2. **寻址**：发送器发送目标设备的地址（例如 0x50），后跟 R/W 位（例如 W 表示写操作）。
3. **数据传输**：发送器发送数据字节，每个字节后接收器发送 ACK 应答位。
- 4.

10.I2C能挂载多少从机设备？

- **7位地址模式(多数情况)**：在标准的I2C总线规范中，每个从机设备都有一个**7位的唯一地址**。因此，一个I2C总线理论上最多可以挂载 $2^7=128$ 个从机设备（地址范围从0x00至0x7F）。然而，由于I2C地址全0（0x00）通常用作广播地址，所以实际可用的从机设备地址数量是 $2^7-1=127$ 个。
- **10位地址模式**：为了支持更多从机设备，I2C协议还定义了**10位地址模式**。在10位地址模式下，地址由10位组成，因此理论上最多可以挂载 $2^{10}=1024$ 个从机设备（地址范围从0x000至0x3FF）。

11.IIC是全双工还是半双工

I2C总线是一种半双工的，主机可以向从机发送数据，然后等待从机的响应，并且这期间从机不能主动发送数据。只有当主机发送完数据并停止传输时，从机才可以主动发送数据。

12.I2C的传输速率是多少？

I²C（Inter-Integrated Circuit）总线协议支持不同的通信速率：

1. **标准模式**：100 kbps //像I2C屏一般是标准模式
2. **快速模式**：400 kbps
3. **高速模式**：3.4 Mbps
4. **超快速模式**：5 Mbps

13.I2C的起始信号、应答信号、停止信号、空闲状态?

起始信号：当SCL为高期间，SDA由高到低的跳变

应答信号：应答位是一个低电平脉冲

停止信号：当SCL为高期间，SDA由低到高的跳变;

空闲状态：当总线上的SDA和SCL两条信号线同时处于高电平,便是空闲状态, 当不传输数据时,SDA和SCL被上拉电阻拉高,即进入空闲状态

14.I2C的读写操作

1.写操作

刚开始主芯片要发出一个**start信号**,然后**发出设备地址(7bit)**(用来确定是往哪一个芯片写数据),**方向位**(读/写, 0表示写, 1表示读)。**等待回应**(用来确定这个设备是否存在), 然后就可以**传输数据** (8bit) , 传输数据之后, 要**等待回应信号**(确定数据是否接受完成, 1位), 然后再传输下一个数据。**每传输一个数据, 接受方都会有一个回应信号**, 数据发送完之后, **主芯片就会发送一个停止信号**。

2.读操作

刚开始主芯片要发出一个**start信号**,然后发出**设备地址 (7bit)** (用来确定是从哪一个芯片读取数据), **方向位**(读/写, 0表示写, 1表示读)。**等待回应**(用来确定这个设备是否存在), 然后就可以**传输数据** (8bit) , 传输数据之后, 要有一个**回应信号**(确定数据是否接受完成), 然后在传输下一个数据。每传输一个数据, 接受方都会有一个回应信号, 数据发送完之后, 主芯片就会发送一个**停止信号**。

15.SPI协议介绍

一、协议特点

- **高速性**：SPI协议能够提供较高的数据传输速率，一般实现通常能达到甚至超过10 Mbps。
- **全双工**：支持同时的数据发送和接收，提高了通信效率。
- **同步通信**：通过时钟信号（SCK）来同步数据的发送和接收。
- **简洁性**：在芯片的管脚上只占用四根线（SCK、MOSI、MISO、SS），节约了芯片的管脚和PCB的布局空间。

二、物理连接

SPI协议使用四条信号线进行通信：

- **SCK (Serial Clock)**：时钟信号线，由主设备产生，用于同步数据传输。
- **MOSI (Master Output, Slave Input)**：主设备输出/从设备输入引脚，用于主设备向从设备发送数据。
- **MISO (Master Input, Slave Output)**：主设备输入/从设备输出引脚，用于从设备向主设备发送数据。
- **SS (Slave Select) /CS (Chip Select)**：从设备选择信号线，也称为片选线，由主设备控制，用于选择与之通信的从设备。

三、通信过程

SPI通信过程通常由主设备发起和控制，具体步骤如下：

1. **选择从设备**：主设备通过将SS/CS线拉低来选择要通信的从设备。
2. **发送时钟信号**：主设备产生SCK时钟信号，用于同步数据的发送和接收。

3. **数据交换**：在SCK的每个时钟周期内，主设备通过MOSI线向从设备发送一位数据，同时从设备通过MISO线向主设备发送一位数据，实现数据的全双工传输。
4. **通信结束**：通信完成后，主设备将SS/CS线拉高，表示结束与当前从设备的通信。

四、通信模式

SPI协议**支持四种通信模式**，这些模式通过时钟极性（CPOL）和时钟相位（CPHA）的不同组合来定义：

- **模式0（CPOL=0, CPHA=0）**：空闲时SCK为低电平，数据在SCK的上升沿采样，下降沿发送。
- **模式1（CPOL=0, CPHA=1）**：空闲时SCK为低电平，数据在SCK的下降沿采样，上升沿发送。
- **模式2（CPOL=1, CPHA=0）**：空闲时SCK为高电平，数据在SCK的下降沿采样，上升沿发送。
- **模式3（CPOL=1, CPHA=1）**：空闲时SCK为高电平，数据在SCK的上升沿采样，下降沿发送。

16.I2C和SPI的区别？

1. **信号线数量**：I2C使用两根线（SDA和SCL），而SPI通常使用四根线（SCK、MOSI、MISO、SS/CS）。
2. **通信方式**：I2C是半双工，SPI是全双工。
3. **主从关系**：I2C支持多主机和多从机，SPI通常是一主多从。
4. **通信速度**：SPI通常比I2C快，但具体速度取决于设备规格。
5. **寻址方式**：I2C通过地址寻址，SPI通过SS/CS线选择从设备。
6. **协议复杂性**：I2C协议相对复杂，包含起始、停止信号和仲裁机制；SPI协议较简单。

17.什么是中断

中断是指在主程序运行过程中，出现了特定的中断触发条件（中断源），使得CPU暂停当前正在运行的程序，转而去处理中断程序。中断处理完成后，CPU会返回到原来被中断的程序执行点，继续执行后续的程序。

18.描述一下中断的执行流程？

CPU正在执行当前程序，由于某个事件的发生产生了中断请求，导致CPU暂停当前程序执行，中断请求会经过中断通道到达NVIC,NVIC去调用专门的中断服务函数执行，执行完毕返回当前程序继续执行。

19.简述一下STM32中断有哪些？

外部中断、定时器中断、DMA中断、UART中断、SPI中断、I2C中断、USB中断、ADC中断。

五、FreeRTOS

1.裸机开发和rtos开发的区别？

裸机开发：

直接在硬件上编程，没有操作系统。

控制完全由程序员决定。

适合简单的实时控制任务。

RTOS开发：

使用实时操作系统（RTOS）进行编程。

支持多任务调度和优先级。

适用于复杂的应用和实时要求高的场景。

总结

裸机开发：直接控制硬件，适合简单的实时控制。

RTOS开发：使用操作系统管理任务，适合复杂的实时应用。

2.FreeRTOS的四种工作状态？

1. **运行状态** (Running)
2. **就绪状态** (Ready)
3. **阻塞状态** (Blocked)
4. **挂起状态** (Suspended)

3.互斥锁与信号量的区别？

互斥锁：

用于实现线程的互斥访问。值只能为0/1。

加锁和解锁必须由同一线程完成。

保证资源的独占访问。

信号量：

用于实现线程的同步。值可以为非负整数。

可以由一个线程释放，另一个线程获取。

支持多个同类资源的同步访问。

总结

互斥锁：保证资源的独占访问。

信号量：支持多线程的同步访问

4.互斥信号量和二值信号量的区别是什么？

互斥信号量是一种特殊的二值信号量，互斥信号量的申请与释放是要在同一个任务中进行的，不能在一个任务中申请而在另一个任务中释放。二值信号量允许在一个任务中申请，在另外一个任务中释放。

5.队列和信号量的区别是什么？

队列：

用于在任务之间传递数据。

支持不同类型的数据结构。

可以传递任意大小的消息。

通常用于数据交换。

信号量：

用于同步任务间的执行顺序。

不传递数据，只传递信号。

支持计数信号量、互斥信号量和二值信号量。

用于控制对共享资源的访问。

总结

队列：用于任务间的数据传递。

信号量：用于任务间的同步控制。

6.任务切换的方式有哪些？

任务切换有两种方式：系统节拍器中断、调用portYIELD，但是，最终都是产生PendSV中断。

7.FreeRTOS中的调度策略是什么？

FreeRTOS使用**抢占式调度策略**。这意味着如果有更高优先级的任务变得可运行（例如，它刚刚完成了一个等待操作），当前运行的任务将被中断，调度器将切换到更高优先级的任务。这确保了关键任务能够及时响应。

8. FreeRTOS如何管理内存？

FreeRTOS管理内存主要通过两种方式：

1. **动态内存分配：**FreeRTOS提供了多种堆内存管理算法（如heap_1到heap_5），允许程序在运行时根据需要动态地申请和释放内存。每种算法有不同的特点和适用场景，如heap_1仅支持分配不支持释放，heap_4则支持合并空闲内存块以减少碎片。
2. **静态内存分配：**对于系统资源（如任务、信号量等），FreeRTOS也支持在编译时静态地分配内存空间，这些资源在RAM中的自定义内存区域创建并绑定。

9.FreeRTOS中的信号量有哪些类型？

FreeRTOS中有**三种主要类型的信号量**：**二值信号量**、**计数信号量**和**递归信号量**。

二值信号量用于锁定资源，类似于互斥锁，它们只能有两个状态：获取或释放。

计数信号量可以用于管理多个相同的资源，或者用来同步多个任务。

递归信号量是计数信号量的一种特殊形式，允许同一个任务多次获取同一个资源。

10.在FreeRTOS中，(IPC)任务间通信有哪些方式？

FreeRTOS支持多种任务间通信机制。最常见的包括使用**队列**、**信号量**、**事件组**和**流缓冲区**。

队列: 用于发送和接收消息；

信号量: 用于同步和互斥；

事件组: 用于表示一组事件的状态；

流缓冲区: 则允许任务通过类似环形缓冲区的结构发送和接收数据流。

六、C++常见问题

1.extern "c"的作用是什么？

extern "C" 的主要作用就是为了能够正确实现C++代码调用其他C语言代码。加上extern "C"后，会指示编译器这部分代码按C语言的进行编译，而不是C++的。

2.new/delete与malloc/free的区别是什么？

1.**new/delete**是C++中的**操作符**，而**malloc/free**是**标准库函数**。

2.对于**非内部数据对象**来说，只使用**malloc**是**无法完成动态对象要求**的，一般在创建对象时需要调用构造函数，对象消亡时，自动的调用析构函数。而**malloc/free**是库函数而不是运算符,不在编译器控制范围之内，不能够自动调用构造函数和析构函数。而**new**在为对象申请分配内存空间时，可以自动调用构造函数，同时也可以完成对对象的初始化。同理，**delete**也可以自动调用析构函数。而**malloc**只是做一件事，只是为变量分配了内存，同理，**free**也只是释放变量的内存。

3.**new**返回的是**指定类型的指针**，并且可以自动计算所申请内存的大小。而 **malloc**需要我们计算申请内存的大小，并且在**返回时强行转换为实际类型的指针**

3.C++中析构函数的作用

如果构造函数打开了一个文件，最后不需要使用时文件就要被关闭。析构函数允许类自动完成类似清理工作，不必调用其他成员函数。析构函数也是特殊的类成员函数。简单来说，析构函数与构造函数的作用正好相反，它用来完成对象被删除前的一些清理工作，也就是专门的扫尾工作。

4.重载和覆盖有什么区别

1.覆盖是子类和父类之间的关系,垂直关系;重载同一个类之间方法之间的关系，是水平关系。

2.覆盖只能由一个方法或者只能由一对方法产生关系;重载是多个方法之间的关系。

3.覆盖是根据对象类型(对象对应存储空间类型)来决定的;而重载关系是根据调用的实参表和形参表来选择方法体的

5.C++中类成员的访问权限？

C++通过 **public**、**protected**、**private** 三个关键字来控制成员变量和成员函数的访问权限，它们分别表示公有的、受保护的、私有的，被称为成员访问限定符。**在类的内部**(定义类的代码内部)，无论成员被声明为 public、protected 还是 private,都是可以互相访问的，**没有访问权限的限制**。**在类的外部**(定义类的代码之外)，**只能通过对象访问成员**，并且通过对象只能访问 public 属性的成员，**不能访问private、protected 属性的成员**

6.C++中struct和class的区别是什么？

在C++中，可以用struct和class定义类,都可以继承。区别在于：

struct的默认继承权限和默认访问权限是public，而class的默认继承权限和默认访问权限是private。

另外，class还可以定义模板类形参比如template.

7. 什么是右值引用，跟左值又有什么区别？

左值和右值的概念

左值: 能对表达式取地址、或具名对象/变量。一般指表达式结束后依然存在的持久对象。

右值: 不能对表达式取地址，或匿名对象。一般指表达式结束就不再

存在的临时对象。

右值引用和左值引用的区别:

- 1.左值可以寻址，而右值不可以。
- 2.左值可以被赋值，右值不可以被赋值，可以用来给左值赋值。
- 3.左值可变,右值不可变(仅对基础类型适用,用户自定义类型右值引用可以通过成员函数改变)。

8.什么是浅拷贝，什么是深拷贝？

深拷贝是彻底的拷贝，两对象中所有的成员都是独立的一份，而且，成员对象中的成员对象也是独立份。

浅拷贝中的某些成员变量可能是共享的，深拷贝如果不够彻底，就是浅拷贝。

9.基类的构造函数/析构函数是否能被派生类继承？

基类的构造函数/析构函数不能被派生类继承。

基类的构造函数不能被派生类继承，派生类中需要声明自己的构造函数。设计派生类的构造函数时,不仅要考虑派生类所增加的数据成员初始化，也要考虑基类的数据成员的初始化。声明构造函数时，只需要对本类中新增成员进行初始化，对继承来的基类成员的初始化，需要调用基类构造函数完成。

基类的析构函数也不能被派生类继承，派生类需要自行声明析构函数。声明方法与一般(无继承关系时)类的析构函数相同，不需要显式地调用基类的析构函数，系统会自动隐式调用。需要注意的是，析构函数的调用次序与构造函数相反。

10.C++如何实现多态？

C++中通过虚函数实现多态。虚函数的本质就是通过基类指针访问派生类定义的函数。每个含有虚函数的类，其实例对象内部都有一个虚函数表指针。该虚函数表指针被初始化为本类的虚函数表的内存地址。所以，在程序中，不管对象类型如何转换，该对象内部的虚函数表指针都是固定的，这样才能实现动态地对对象函数进行调用，这就是C++多态性的原理。

11.纯虚函数是什么？

- **定义：**声明时不提供实现的虚函数。
- **语法：**使用 `= 0` 标记。
- **用途：**强制派生类必须重写该函数。
- **示例：**

```
class Base {  
public:  
    virtual void func() = 0;    // 纯虚函数  
};
```

12.面向对象的三大特征

面向对象的三大特征是：**封装性、继承性和多态性。**

13.QT信号与槽如何进行绑定？

在Qt中，信号与槽的绑定通常通过以下几种方式进行：

1. **自动连接：**

- 使用 `QObject::connect()` 函数直接连接信号和槽。

```
QObject::connect(sender, &SenderClass::signalName,  
receiver, &ReceiverClass::slotName);
```

2. UI设计器:

- 在Qt Designer中, 可以通过拖拽方式将界面元素的信号与槽连接起来。

3. 元对象系统:

- 利用Qt的元对象系统, 在类的构造函数或初始化列表中进行连接。

Cpp深色版本

```
QMetaObject::connectSlotsByName(this);
```

这些方法使得信号能够触发相应的槽函数, 实现对象间的通信。

七、项目描述参考

*PS:项目描述最好按自己的表述去说, 这样会表述得更流畅和清晰。
下面的表述仅作为参考*

1.智慧生活控制系统

该项目基于全志H616 Linux平台开发, 能够实现通过语音控制、人脸识别、网络远程控制各类家电(根据项目名称换个词)外设, 比如像卧室、客厅灯光、风扇、门锁的控制等。同时还能进行火灾警情的实时监控以及在线实时显示功能。

这里面, 项目架构采用简单工厂模式来设计, 将语音识别, socket, 火灾检测, 消息监听设计成链表的每个节点, 形成控制工厂。将卧室灯, 餐厅灯, 客厅灯, 摄像头、iic屏幕等模块也设计成链表中的每个节点, 形成设备端工厂。基于这种架构保证项目的稳

定性和功能拓展性，在添加新功能的时候，只需要添加一个链表节点就可以了。

同时项目涉及到多线程的并发管理，包含控制端的语音、网络、火灾检测三个实时监听线程以及设备处理端的消息接收处理线程。当控制线程比如语音线程监听语音指令后，通过消息队列给消息接收处理线程发送对应的控制指令，消息队列监听线程接收到指令后，创建控制设备处理线程，该线程里通过设备链表里提前定义好的设备信息对相应的外设进行操作，即可实现开关风扇、灯光等操作。如果接收到的消息是报警模块发出的，就驱动蜂鸣器报警，同时给语音模块发送语音报警的指令。设备的操作结果和火灾报警会在Oled屏上显示。

另外还可以通过摄像头采集人脸数据后通过，对人脸进行动态识别检测。如识别成功，操作电磁锁打开对应的房门，5s后又自动关上。

2.智能分拣项目

该项目基于ARM-Linux平台开发，通过语音和网络远程控制，利用绿色资源（可以看自己写什么名称改下名字）的智能图像动态识别技术功能识别出当前物品的类型，根据动态识别出的结果打开不同的箱子进行操作，并通过OLED显示当前的物品类型及进行一个物品的实时播报。

这里面涉及到至少3个线程，语音线程实时监听语音控制指令的输入。网络线程在线监听远程app的接入控制。当接收到正确的分类指令后。利用条件变量触发物品识别在线线程进行物品的动态识别，并根据识别的结果创建不同的线程进行打开箱子操作，语音播报和OLED显示等操作

3.小车项目

该项目基于51单片机/32单片机实现了小车避障、循迹、跟随、测速显示及语音控制、WIFI控制功能。

可通过语音和wifi控制，实现小车当前模式切换（避障、循迹、跟随），并在OLED屏上显示当前模式

利用PWM控制两边的电机，从而实现小车的前进，后退，左转、右转（左右两边的差速控制）功能。

通过TCRT5000传感器工作原理即模块发射的红外线照射到黑线时，红外线将会被黑线吸收的特性实现循迹功能

通过SG90舵机和HC-SR04超声波模块测距从而实现避障功能

使用OLED实时显示当前速度和小车模式、